Mobile Programming

FROM API TO SECURE API — PRESIDENTKU

Mengamankan komunikasi aplikasi mobile dengan token-based authentication.



PRESIDENTKU
Aplikasi mobile mengakses API menggunakan akun pengguna.

↓

LOGIN
Pengguna memasukkan email dan password.

↓

ACCESS TOKEN
Server memvalidasi kredensial dan mengembalikan access token (JWT / Bearer Token).

↓

SECURE REQUEST
Aplikasi mengirim request ke API dengan menyertakan access token pada header (Authorization).

↓

REST API
Server memverifikasi token dan memproses permintaan.

↓

PROTECTED DATA
Data hanya diberikan jika token valid dan pengguna memiliki akses yang sesuai.

Konsep Utama

API yang dapat diakses bukan berarti API yang boleh diakses siapa saja.

Gunakan token-based authentication untuk memastikan hanya pengguna yang terautentikasi dan berwenang yang dapat mengakses data melalui API.

Mobile Programming

NETWORK SECURITY — WHY DOES API NEED PROTECTION?

Mahasiswa memabami risiko ketika API tidak memiliki mekanisme autentikasi.

Alur Sederhana

MOBILE APP
Aplikasi mobile mengakses API.

API
Server memproses permintaan.

DATA
Data dikirimkan ke aplikasi mobile.

Tanpa Proteksi

API dapat diakses siapa saja sehingga data berisiko disalahgunakan.

REQUEST
Permintaan dikirim langsung ke API.

API
Memproses permintaan tanpa validasi pengguna.

DATA
Data dapat diakses oleh siapa saja.

Dengan Authentication

API hanya memberikan data kepada pengguna yang terautentikasi dan memiliki akses yang sesuai.

REQUEST + TOKEN
Permintaan dikirim dengan token (JWT/Bearer Token) pada header.

API
Server memvalidasi token dan mengecek hak akses pengguna.

DATA
Data hanya diberikan jika token valid dan pengguna memiliki akses yang sesuai.

Konsep Utama

API yang dapat diakses bukan berarti API yang boleh diakses siapa saja.

Gunakan mekanisme autentikasi dan kontrol akses untuk memastikan hanya pengguna yang berwenang yang dapat mengakses data melalui API.

Fokus Pembahasan

1. **Authentication**
   Memverifikasi identitas pengguna (misal: login, access token).
2. **Authorization**
   Mengatur hak akses terhadap resource berdasarkan peran atau permission.
3. **Secure communication**
   Menggunakan HTTPS untuk melindungi data saat dikirim.
4. **Protected endpoint**
   Endpoint API dilindungi dengan token dan validasi akses.
5. **Token**
   Menggunakan token (misal JWT) untuk setiap request ke API.

Mobile Programming

SPRINT 04 — SECURE THE PRESIDENTKU API

Mengimplementasikan keamanan akses API pada aplikasi mobile PresidentKu.

Alur Sprint 04

PRESIDENTKU	Aplikasi mobile mengakses API.
↓	
LOGIN	Pengguna memasukkan email dan password.
↓	
TOKEN	Server mengirimkan access token (JWT).
↓	
SECURE REQUEST	Aplikasi mengirim request dengan token pada header.
↓	
PROTECTED API	Server memvalidasi token dan memberikan akses.
↓	
SECURE DATA	Data diterima dan ditampilkan di aplikasi.

Target Sprint 04

Mahasiswa mampu:

1. Memahami token-based authentication.
2. Memahami access token.
3. Memahami JWT.
4. Melakukan login melalui API.
5. Menerima token.
6. Menyimpan token secara aman.
7. Mengirim token pada request.
8. Mengakses protected endpoint.
9. Mengakhiri session.
10. Mempertahankan keamanan dari Pekan 3.

Mobile Programming

TOKEN-BASED AUTHENTICATION — WHO CAN ACCESS THE API?

Memahami cara kerja token-based authentication dan siapa yang dapat mengakses API.

Proses Authentication (Login)

1. **USER**
   Pengguna memasukkan email dan password pada aplikasi.
2. **LOGIN**
   Aplikasi mengirimkan data login ke server melalui API.
3. **SERVER**
   Server memverifikasi kredensial pengguna.
4. **TOKEN**
   Server membuat dan mengirimkan token (JWT / Bearer Token) sebagai bukti autentikasi.
5. **MOBILE APP**
   Aplikasi menerima token dan menyimpannya secara aman.

Proses Mengakses API (Dengan Token)

1. **TOKEN**
   Aplikasi menggunakan token yang tersimpan.
2. **REQUEST**
   Aplikasi mengirimkan request ke API dengan token pada header (Authorization: Bearer <token>).
3. **SERVER**
   Server menerima request dan memvalidasi token (cek keabsahan, masa berlaku, dan hak akses).
4. **VALIDATE TOKEN**
   Jika token valid, server memberikan akses ke resource yang diminta.
5. **RESPONSE**
   Server mengirimkan data response dalam format JSON.

Konsep Inti

Token adalah bukti bahwa client telah berhasil melakukan authentication.

Token digunakan oleh client (mobile app) untuk membuktikan identitas dan **hak akses** saat mengakses API. Tanpa token yang valid, API tidak akan memberikan akses ke resource yang dilindungi.

Fokus Pembahasan

1. **Authentication**
   Proses verifikasi pengguna.
2. **Token**
   Bukti autentikasi (contoh: JWT / Bearer Token).
3. **Client**
   Aplikasi mobile yang mengirim request ke API.
4. **Server**
   Memvalidasi token dan mengatur akses.
5. **Protected resource**
   Endpoint API yang hanya dapat diakses dengan token yang valid.

Mobile Programming

ACCESS TOKEN — THE KEY TO PROTECTED API

Token yang digunakan untuk mengakses endpoint API yang dilindungi.

Alur Penggunaan Access Token

1. **LOGIN**
   Pengguna memasukkan email dan password.
2. **ACCESS TOKEN**
   Server mengirimkan access token (biasanya JWT).
3. **API REQUEST**
   Aplikasi mengirim request ke API dengan token pada header Authorization.
4. **AUTHORIZED**
   Server memvalidasi token dan memberikan akses ke protected endpoint.

Contoh Penggunaan di HTTP Request

```
GET /api/programs HTTP/1.1
Host: api.presidentku.id
Authorization: Bearer eyJhbGciOiJIUz...
Accept: application/json
```

Penjelasan

Access Token
Token yang diberikan setelah login untuk mengakses resource yang dilindungi.

Bearer Token
Skema autentikasi yang mengirimkan token langsung pada header Authorization dengan format Bearer <token>.

Authorization Header
Header HTTP yang digunakan untuk mengirim token pada setiap request ke API.

Protected Endpoint
Endpoint API yang hanya dapat diakses jika token valid dan memiliki permission yang sesuai.

Konsep Utama

Access token adalah kunci untuk mengakses API yang dilindungi.

Access token digunakan oleh client (mobile app) untuk membuktikan identitas dan hak akses pada setiap request ke API.

Fokus Pembahasan

1. **Access Token**
   Pengertian dan fungsi access token.
2. **Bearer Token**
   Cara kerja dan penggunaan bearer token.
3. **Authorization Header**
   Format dan implementasi pada HTTP request.
4. **Protected Endpoint**
   Endpoint yang membutuhkan token valid.

Pertanyaan Penting

Mengapa token dikirim bersama request?
Karena server perlu mengetahui apakah request tersebut berasal dari client yang telah terautentikasi.

Mobile Programming

JWT — UNDERSTANDING THE TOKEN

JSON Web Token (JWT) adalah salah satu implementasi token-based authentication.

Alur JWT Authentication

1. **LOGIN**
   Pengguna memasukkan email dan password.
2. **JWT**
   Server mengirimkan JWT setelah login berhasil.
3. **CLIENT**
   Aplikasi menyimpan JWT secara aman.
4. **API REQUEST**
   Aplikasi mengirim request dengan JWT pada header Authorization.
5. **SERVER VALIDATION**
   Server memvalidasi JWT dan memberikan akses jika token valid.

Struktur JWT

HEADER
Informasi tipe token dan algoritma.

+

PAYLOAD
Data (claims) seperti user `id`, `role`, dan expired time.

+

SIGNATURE
Verifikasi integritas token menggunakan secret key.

JWT
`header.payload.signature`
(string yang dipisahkan dengan tanda titik)

Contoh Struktur JWT (Decoded)

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Menunjukkan tipe token dan algoritma yang digunakan.

Payload

```
{
  "sub": "1",
  "name": "User",
  "role": "student",
  "exp": 1728144000
}
```

Berisi data (claims) seperti user id, role, dan waktu kadaluarsa (expiration).

Signature

```
H30e2...X9fQ
```

Digunakan untuk verifikasi integritas token. Dibuat menggunakan secret key di server.

Konsep Utama

JWT adalah token dalam format string yang berisi informasi pengguna dan ditandatangani secara digital.

JWT digunakan untuk autentikasi pada API yang bersifat stateless, sehingga server dapat memvalidasi token tanpa perlu menyimpan session di server.

Poin Pembahasan

1. **Header**
   Berisi tipe token dan algoritma (misal: HS256).
2. **Payload**
   Berisi data pengguna (claims) seperti user id, role, dan expiration.
3. **Signature**
   Digunakan untuk memastikan token tidak dimanipulasi.
4. **Token validation**
   Server memverifikasi signature, masa berlaku, dan hak akses.
5. **Expiration**
   Token memiliki masa berlaku (exp) untuk meningkatkan keamanan.

Mobile Programming

HTTPS — PROTECT THE DATA IN TRANSIT

Memahami pentingnya HTTPS untuk melindungi data yang dikirim antara aplikasi mobile dan server.

Konsep Utama

Token yang aman disimpan tetapi dikirim melalui koneksi yang tidak aman tetap menjadi masalah.

HTTPS mengenkripsi data yang dikirim antara aplikasi mobile dan server sehingga tidak mudah disadap atau diubah oleh pihak yang tidak berwenang.

Poin Pembahasan

1. **HTTP**
   Protokol tanpa enkripsi, data dapat disadap.
2. **HTTPS**
   Protokol dengan enkripsi menggunakan TLS.
3. **Data in transit**
   Data yang dikirim melalui jaringan.
4. **Secure connection**
   Koneksi terenkripsi untuk melindungi data.
5. **Certificate / TLS**
   Sertifikat digital untuk memverifikasi server dan mengenkripsi komunikasi.

Perbandingan HTTP vs HTTPS

HTTP

Data dikirim dalam bentuk teks biasa (tidak terenkripsi) sehingga berisiko disadap.



HTTPS

Data dikirim dalam bentuk terenkripsi menggunakan TLS sehingga lebih aman.



Alur Komunikasi HTTPS



Ilustrasi TLS (Sederhana)



Mobile Programming

TOKEN STORAGE & SECURE INTERCEPTOR — MANAGE YOUR TOKEN SAFELY

Menyimpan token dengan aman dan mengirimkannya secara otomatis pada setiap request.

Alur Manajemen Token

1. **LOGIN**
   Pengguna memasukkan email dan password.
2. **ACCESS TOKEN**
   Server mengirimkan access token (dan refresh token).
3. **SECURE STORAGE**
   Token disimpan secara aman di perangkat (secure storage).
4. **API REQUEST**
   Aplikasi membuat request ke API.
5. **INTERCEPTOR**
   Interceptor menambahkan token pada setiap request.
6. **AUTHORIZATION HEADER**
   Token dikirim melalui header: Authorization: Bearer <token>.
7. **API**
   Server memvalidasi token dan memberikan akses ke endpoint.

Mekanisme Interceptor

Interceptor bekerja otomatis untuk menambahkan token pada setiap request ke API, sehingga tidak perlu menulis header token di setiap API call.

Contoh Interceptor (Konseptual)

```
// Contoh konsep interceptor (misal dengan Axios)
api.interceptors.request.use(async (config) => {
  const token = await getToken(); // dari secure storage
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Access Token vs Refresh Token

Access Token

- Digunakan untuk mengakses protected endpoint.
- Memiliki masa berlaku (expired).
- Disimpan secara aman di perangkat.
- Dikirim melalui Authorization header.

Refresh Token

- Digunakan untuk mendapatkan access token baru.
- Masa berlaku lebih panjang.
- Disimpan secara aman di perangkat.
- Mekanisme refresh token rotation dapat dipelajari pada pekan berikutnya.

Konsep Utama

Token harus disimpan dengan aman dan dikirim secara otomatis dengan interceptor.

Penyimpanan token yang aman mencegah kebocoran data, dan interceptor memastikan token selalu dikirim pada setiap request ke API tanpa perlu menulis kode berulang.

Fokus Pembahasan

1. **Token storage**
   Cara menyimpan token secara aman di perangkat.
2. **Access token**
   Fungsi dan masa berlaku access token.
3. **Refresh token**
   Konsep dan perannya dalam memperbarui access token.
4. **Interceptor**
   Menambahkan token secara otomatis pada setiap request.
5. **Authorization header**
   Format dan penggunaan header Authorization.

Catatan Penting

Token yang aman disimpan tetapi dikirim melalui koneksi yang tidak aman tetap menjadi masalah.

Selalu gunakan HTTPS untuk melindungi token saat dikirim ke server.

Mobile Programming

CODE EXAMPLE — LOGIN + STORE ACCESS TOKEN

Contoh potongan kode untuk login dan menyimpan access token secara aman.

Alur Proses

1. **LOGIN**
   Pengguna memasukkan email dan password.
2. **API**
   Aplikasi mengirim request ke endpoint /login.
3. **TOKEN**
   Server mengirim access token (dalam response JSON).
4. **SECURE STORE**
   Access token disimpan secara aman di perangkat menggunakan SecureStore.

Contoh Potongan Kode (React Native + Fetch)

```
// 1. Login ke API
const response = await fetch(
  "/login",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      email,
      password,
    }),
  }
);

const data = await response.json();

// 2. Simpan access token ke secure storage
await SecureStore.setItemAsync(
  "access_token",
  data.access_token
);
```

Konsep Utama

Login menghasilkan token yang kemudian digunakan untuk request berikutnya.

Setelah pengguna berhasil login, server akan mengirim access token. Token ini disimpan secara aman di perangkat dan digunakan pada setiap request ke endpoint yang dilindungi.

Penjelasan Kode

1. **Mengirim request POST ke endpoint /login**
   Menggunakan email dan password.
2. **Menerima response dari server**
   Server mengembalikan data dalam format JSON yang berisi access token.
3. **Menyimpan access token**
   Token disimpan menggunakan SecureStore agar lebih aman dibandingkan AsyncStorage.

Hal yang Perlu Diperhatikan

- **Jangan menyimpan token di plain storage**
  Gunakan SecureStore untuk menyimpan token secara aman di perangkat.
- **Token digunakan untuk request berikutnya**
  Access token akan dikirim melalui header Authorization pada setiap request ke API.

Mobile Programming

TASK 01 — IMPLEMENT TOKEN AUTHENTICATION

Tambahkan authentication berbasis API ke aplikasi PresidentKu.

Alur Task

1. **LOGIN**
   Pengguna memasukkan email dan password.
2. **API**
   Aplikasi mengirim request login ke endpoint API.
3. **ACCESS TOKEN**
   Server mengirim access token dalam response JSON.
4. **SECURE STORAGE**
   Token disimpan secara aman di perangkat (SecureStore).
5. **AUTHENTICATED**
   Aplikasi menyimpan authentication state dan siap mengakses endpoint lainnya.

Implementasi di PresidentKu



Contoh Endpoint Login

```
POST /login
Request Body:
{
  "email": "user@mail.com",
  "password": "password123"
}
```

Response (contoh):

```
{
  "access_token":
  "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 1,
    "name": "User"
  }
}
```

Tujuan Task

Mahasiswa mampu mengimplementasikan authentication berbasis API pada aplikasi PresidentKu sesuai dengan konsep token-based authentication.

Yang Harus Dibuat Mahasiswa

- **Login request**
  Mengirim request ke endpoint login.
- **Credential validation**
  Mengirim email dan password ke server.
- **Token retrieval**
  Menerima access token dari response.
- **Secure token storage**
  Menyimpan token menggunakan SecureStore.
- **Authentication state**
  Mengatur state aplikasi ketika user sudah login.

Checklist Task

- [ ] Login dikirim ke API
- [ ] Response diterima
- [ ] Access token diperoleh
- [ ] Token disimpan secara aman
- [ ] Authentication state berubah

Mobile Programming

TASK 02 — ATTACH TOKEN TO API REQUEST

Gunakan token yang sudah disimpan untuk mengakses protected API pada PresidentKu.

Alur Proses

1. **SECURE STORAGE**
   Ambil token yang sudah disimpan di perangkat (SecureStore).
2. **GET TOKEN**
   Membaca access token dari secure storage.
3. **API REQUEST**
   Mengirim request ke endpoint yang dilindungi (protected).
4. **AUTHORIZATION**
   Menambahkan token pada header Authorization: Bearer <token>.
5. **PROTECTED API**
   Server memvalidasi token dan mengembalikan data PresidentKu.

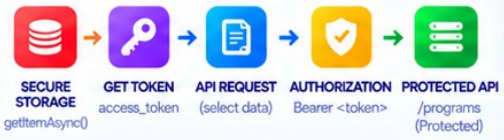
Contoh Potongan Kode (React Native + Fetch)

```
// 1. Ambil access token dari SecureStore
const token = await SecureStore.getItemAsync(
  "access_token",
);

// 2. Kirim request ke protected endpoint
const response = await fetch(
  "https://api.presidentku.id/programs",
  {
    method: "GET",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
  }
);

// 3. Proses response
const data = await response.json();
console.log(data);
```

Ilustrasi Request ke Protected API



Contoh Tampilan Data dari Protected API



Tujuan Task

Mahasiswa mampu menggunakan token yang telah disimpan untuk mengakses data dari protected API dan menampilkannya di aplikasi PresidentKu.

Yang Harus Dibuat Mahasiswa

- **Ambil token dari SecureStore**
  Membaca access token yang sudah disimpan pada Task 01.
- **Kirim request ke protected API**
  Menambahkan header Authorization: Bearer <token>.
- **Terima dan proses response**
  Mengambil data dari endpoint yang dilindungi.
- **Tampilkan data di aplikasi**
  Menampilkan data PresidentKu (misal: daftar program).

Checklist Task

- [ ] Token berhasil dibaca dari SecureStore
- [ ] Request dikirim dengan header Authorization
- [ ] Response berhasil diterima
- [ ] Data ditampilkan di aplikasi
- [ ] Aplikasi hanya bisa mengakses data melalui protected API

Mobile Programming

TASK 03 — PROTECT THE PRESIDENTKU SESSION

Kelola session pengguna dengan aman mulai dari login, penggunaan token, hingga logout.

Alur Session

1. **LOGIN**
   Pengguna memasukkan email dan password ke API.
2. **TOKEN**
   Server mengirim access token yang disimpan secara aman di SecureStore.
3. **SESSION**
   Aplikasi menyimpan session dan menggunakan token untuk request berikutnya.
4. **PROTECTED API**
   Setiap request ke endpoint terlindungi menggunakan token (Authorization header).
5. **LOGOUT**
   Token dihapus, session dibersihkan, dan aplikasi kembali ke kondisi unauthenticated.

Ilustrasi Alur Penggunaan Session di Aplikasi



1. **Login** — Token diperoleh
2. **App digunakan** — Token digunakan untuk request ke protected API
3. **Logout** — Token dihapus dan session dibersihkan
4. **Unauthenticated** — Aplikasi kembali ke halaman login

Contoh Kode Logout (Hapus Token)

```
// Hapus access token dari SecureStore
await SecureStore.deleteItemAsync("access_token");

// Bersihkan state authentication
setIsAuthenticated(false);

// Navigasi ke halaman login
router.replace("/login");
```

Perubahan State Aplikasi

- **Login** — isAuthenticated: true
- **App digunakan** — Token dipakai untuk API request
- **Logout** — Token dihapus, isAuthenticated: false
- **Unauthenticated** — Kembali ke halaman login

Tujuan Task

Mahasiswa mampu mengelola session pengguna secara aman pada aplikasi PresidentKu mulai dari login, penggunaan token, hingga logout.

Yang Harus Dibuat Mahasiswa

- **Login berhasil**
  Token diperoleh dari API dan disimpan menggunakan SecureStore.
- **Gunakan token**
  Token digunakan otomatis pada setiap request ke protected API.
- **Implementasi logout**
  Menghapus token dan membersihkan session dari aplikasi.
- **Pertahankan fitur sebelumnya**
  Accessibility (Pekan 2) dan secure storage (Pekan 3) tetap digunakan.

Checklist Task

- [ ] Login berhasil dan token diperoleh
- [ ] Token disimpan dengan SecureStore
- [ ] Token digunakan untuk request ke protected API
- [ ] Logout menghapus token
- [ ] Session dibersihkan
- [ ] Aplikasi kembali ke halaman login
- [ ] Accessibility dari Pekan 2 tetap diterapkan
- [ ] Secure storage dari Pekan 3 tetap digunakan

Mobile Programming

DEMO — SHOW YOUR SECURE API INTEGRATION

Demonstrasikan implementasi authentication dengan protected API pada aplikasi PresidentKu.

Alur Demo

1. **RUN** — Jalankan aplikasi PresidentKu.
2. **LOGIN** — Masukkan email dan password.
3. **TOKEN** — Token diterima dari server.
4. **API REQUEST** — Request ke protected API dengan token.
5. **PROTECTED DATA** — Response Data API tampil secara dari protected endpoint.
6. **LOGOUT** — Token dihapus dan session berakhir.



Yang Harus Ditunjukkan

1. **Login PresidentKu** — Masukkan email dan password.
2. **Token diterima** — Tampilkan bahwa token berhasil diperoleh.
3. **Token tersimpan** — Buktikan token tersimpan secara aman (misal cek di SecureStore).
4. **Request protected dilakukan** — Gunakan token pada header Authorization.
5. **Data API tampil** — Tampilkan data dari protected endpoint.
6. **Logout** — Hapus token dan akhiri session.
7. **Session berakhir** — Aplikasi kembali ke halaman login.

Contoh Log / Console

```
[1] App started
[2] Login request → POST /login
[3] Response: { access_token: "eyJhbGciOi..." }
[4] Token saved to SecureStore
[5] Request → GET /programs
[6] Headers: Authorication: Bearer eyJhbGciOi..."
[7] Response: 200 OK (protected data)
[8] Logout → token deleted
[9] Navigate to login screen
```

Contoh Data di SecureStorage

SecureStore.getItemAsync("access_token")

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "expires_at": "2026-10-12T08:30:00Z"
}
```

Token disimpan menggunakan SecureStore (secara aman di perangkat).

Contoh Response Protected API

```
GET /programs
Headers:
 Authorization: Bearer eyJhbGciOi..."
Response 200 OK
[
  {
    "id": 1,
    "title": "Indonesia Maju",
    "description": "Program ...",
    "image": "https://...",
  }
]
```

Pertanyaan Demo

1. Dari mana access token diperoleh?
2. Di mana token disimpan?
3. Bagaimana token dikirim ke API?
4. Apa fungsi Bearer Token?
5. Apa yang terjadi jika token tidak dikirim?
6. Apa yang terjadi ketika logout?

Mobile Programming

REVIEW — FROM PUBLIC API TO PROTECTED API

Membandingkan implementasi sebelum dan sesudah menggunakan authentication pada PresidentKu.

CURRENT

Tanpa Authentication
(Mengakses Public API)

- **PresidentKu** (Aplikasi)
- ↓
- **REST API** (Public Endpoint)
- ↓
- **DATA** (Tanpa perlu login)

REVIEW

Konsep Authentication
(Login → Token → Authorization → API)

- **LOGIN** Kirim email dan password ke API
- ↓
- **TOKEN** Terima access token dari server
- ↓
- **AUTHORIZATION** Kirim token pada header Authorization
- ↓
- **API** Akses endpoint menggunakan token

IMPROVED

PresidentKu dengan Protected API
(Aplikasi lengkap menggunakan authentication)

- **PresidentKu** (Aplikasi)
- ↓
- **LOGIN** Email & Password
- ↓
- **ACCESS TOKEN** Diterima dari server
- ↓
- **SECURE REQUEST** Kirim request dengan Authorization header
- ↓
- **PROTECTED API** (Requires token)
- ↓
- **DATA** (Protected content)

Review Checklist

- [ ] Authentication berhasil
- [ ] Access token diterima
- [ ] Token disimpan secara aman
- [ ] Authorization header digunakan
- [ ] Protected endpoint dapat diakses
- [ ] Request tanpa token ditolak
- [ ] Logout menghapus token
- [ ] HTTPS digunakan
- [ ] Secure storage dari Pekan 3 tetap digunakan

Key Takeaway

Security
Protected API memastikan data hanya dapat diakses oleh pengguna yang telah login.

Token
Access token digunakan untuk mengautentikasi setiap request ke endpoint yang dilindungi.

Secure Storage
Token harus disimpan secara aman menggunakan SecureStore (Pekan 3).

HTTPS
Seluruh komunikasi data harus menggunakan HTTPS untuk melindungi data di dalam transit.

Session Management
Logout harus menghapus token dan mengakhiri session agar aplikasi kembali ke kondisi unauthenticated.

Mobile Programming

SPRINT 04 — SECURE PRESIDENTKU API

Checklist penyelesaian Sprint 04 untuk memastikan implementasi authentication dengan aman pada aplikasi PresidentKu.

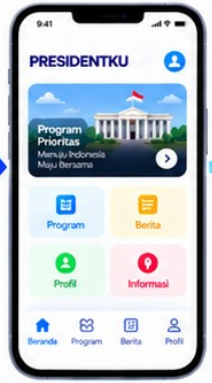


Alur Implementasi Sprint 04

1. Login melalui API
2. Access Token (JWT)
3. Simpan Token secara aman (SecureStore)
4. Kirim request dengan Authorization Bearer Token
5. Akses Protected API
6. Dapatkan data & tampil di aplikasi



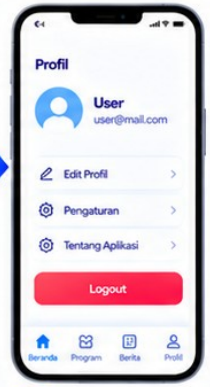
1 Login melalui API



2 Gunakan token untuk protected API



3 Data dari protected endpoint



4 Logout dan hapus session

Contoh Authorization Header

```
const token = await
  SecureStore.getItemAsync(
    "access_token",
);
const response = await fetch(
  "https://api.presidentku.id/programs",
  {
    method: "GET",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${token}`,
    },
  }
);
const data = await response.json();
console.log(data);
```

Sprint 04 Checklist

- [ ] Token-based authentication
- [ ] Login melalui API
- [ ] Access token
- [ ] Bearer token
- [ ] Authorization header
- [ ] JWT concept
- [ ] HTTPS
- [ ] Secure token storage
- [ ] Protected endpoint
- [ ] Token validation
- [ ] Secure API request
- [ ] Logout
- [ ] Session cleared
- [ ] Test authenticated request
- [ ] Test unauthenticated request
- [ ] Test pada Expo Go
- [ ] Commit
- [ ] Push
- [ ] Ready for next sprint